

API Documentation

Table of Contents

- [API Documentation](#)

- [Overview](#) - [CLI Commands](#) - [Command Reference](#) - [Programmatic API](#) - [Core Functions](#) - [Documentation Generation](#) - [AI Client](#) - [Utility Functions](#) - [Changelog Functions](#) - [Diagram Functions](#) - [Screenshot Functions](#) - [Data Structures](#) - [Manifest \(inferred\)](#) - [FunctionDefinition \(inferred\)](#) - [ClassDefinition \(inferred\)](#) - [Error Handling](#) - [Rate Limiting](#) - [Environment Variables](#) - [Configuration File](#) - [Related Documentation](#)

Overview

Athena is a documentation generation tool and does not expose a traditional REST API or web service API. Instead, it provides a **Command-Line Interface (CLI)** and a **Programmatic Node.js API** for generating project documentation. **Base Usage:**

- CLI: `athena [options]`
- Programmatic: `import { scanProject, DocGenerator } from 'athena'`

Authentication: Not applicable (local tool)

CLI Commands

The CLI is the primary interface for Athena. Commands are invoked via the command line.

Command Reference

Command	Description	Status
-----	-----	-----
<code>scan</code>	Scan project and generate documentation	(inferred)
<code>generate</code>	Generate documentation from manifest	(inferred)
<code>freshness</code>	Check documentation freshness	(inferred)
<code>changelog</code>	Generate changelog from git history	(inferred)
<code>diagrams</code>	Generate architecture diagrams	(inferred)

Programmatic API

Core Functions

`scanProject(projectDir: string): Promise`

Scans a project directory and returns a manifest of detected functions, classes, and structure.

Parameters:

Name	Type	Required	Description
-----	-----	-----	-----
<code>projectDir</code>	string	Yes	Absolute or relative path to project root

Returns: `Promise` — Project manifest object **Example:**

```
import { scanProject } from 'athena';

const manifest = await scanProject('./my-project');

console.log(manifest.functions);

console.log(manifest.classes);
```

`loadConfig(projectDir: string): Promise`

Loads Athena configuration from the project directory.

Parameters:

Name	Type	Required	Description
-----	-----	-----	-----
<code>projectDir</code>	string	Yes	Path to project root

Returns: `Promise` — Configuration object **Example:**

```
import { loadConfig } from 'athena';

const config = await loadConfig('./my-project');
```

```
console.log(config);
```

`getConfigPath(projectDir: string): string`

Returns the expected configuration file path for a project.

Parameters:

Name	Type	Required	Description
-----	-----	-----	-----
<code>projectDir</code>	string	Yes	Path to project root

Returns: `string` — Absolute path to config file **Example:**

```
import { getConfigPath } from 'athena';

const configPath = getConfigPath('./my-project');

// Returns: '/absolute/path/to/my-project/.athenarc' (inferred)
```

`hasConfig(projectDir: string): boolean`

Checks if a project has an Athena configuration file.

Parameters:

Name	Type	Required	Description
-----	-----	-----	-----
<code>projectDir</code>	string	Yes	Path to project root

Returns: `boolean` — True if config exists **Example:**

```
import { hasConfig } from 'athena';

if (hasConfig('./my-project')) {

  console.log('Configuration found');
}
```

Documentation Generation

DocGenerator Class

Generates documentation using AI (Claude) from project manifests.

Constructor:

```
new DocGenerator(config?: GeneratorConfig)
```

Methods:

```
##### generate(manifest: Manifest, type: DocType): Promise
```

Generates documentation of specified type from manifest.

Parameters:

Name	Type	Required	Description
-----	-----	-----	-----
manifest	Manifest	Yes	Project manifest from scanner
type	DocType	Yes	Type of documentation to generate

Returns: `Promise` — Generated markdown documentation **Example:**

```
import { DocGenerator, scanProject } from 'athena';

const manifest = await scanProject('./my-project');

const generator = new DocGenerator();

const readme = await generator.generate(manifest, 'README');

const api = await generator.generate(manifest, 'API');
```

```
##### join(sections: string[]): string
```

Joins multiple documentation sections into a single document.

Parameters:

Name	Type	Required	Description
-----	-----	-----	-----

sections	string[]	Yes	Array of markdown sections
----------	----------	-----	----------------------------

Returns: `string` — Combined markdown document **Example:**

```
const generator = new DocGenerator();
const combined = generator.join([overview, apiDocs, examples]);
```

AI Client

ClaudeClient Class

Interfaces with Anthropic's Claude API for AI-powered documentation generation.

Constructor:

```
new ClaudeClient(apiKey: string, model?: string)
```

Methods:

`getModelName(): string`

Returns the currently configured Claude model name.

Returns: `string` — Model identifier (e.g., 'claude-3-5-sonnet-20241022') **Example:**

```
import { ClaudeClient } from 'athena';

const client = new ClaudeClient(process.env.ANTHROPIC_API_KEY);

console.log(client.getModelName());
```

`generate(prompt: string, context?: object): Promise`

Generates content using Claude API based on prompt and context.

Parameters:

Name	Type	Required	Description
-----	-----	-----	-----
<code>prompt</code>	string	Yes	Generation prompt/instructions
<code>context</code>	object	No	Additional context for generation

Returns: `Promise` — Generated content **Example:**

```
const client = new ClaudeClient(process.env.ANTHROPIC_API_KEY);
const documentation = await client.generate(

  'Generate API documentation',
  { manifest, endpoints }
);
```

Utility Functions

`parseMarkdownSections(markdown: string): Section[]`

Parses markdown into structured sections based on headers.

Parameters:

Name	Type	Required	Description
-----	-----	-----	-----
<code>markdown</code>	<code>string</code>	Yes	Markdown content to parse

Returns: `Section[]` — Array of section objects

`slugify(text: string): string`

Converts text to URL-safe slug format.

Parameters:

Name	Type	Required	Description
-----	-----	-----	-----
<code>text</code>	<code>string</code>	Yes	Text to slugify

Returns: `string` — Slugified text **Example:**

```
import { slugify } from 'athena';

const slug = slugify('API Documentation'); // 'api-documentation'
```

generateTableOfContents(sections: Section[]): string

Generates markdown table of contents from sections.

Parameters:

Name	Type	Required	Description
-----	-----	-----	-----
<code>sections</code>	<code>Section[]</code>	Yes	Parsed markdown sections

Returns: `string` — Markdown TOC

buildSourceSummary(manifest: Manifest): string

Creates a summary of source code structure from manifest.

Parameters:

Name	Type	Required	Description
-----	-----	-----	-----
<code>manifest</code>	<code>Manifest</code>	Yes	Project manifest

Returns: `string` — Formatted summary text

buildCrossReferences(docs: Document[]): Map

Builds cross-reference links between documentation sections.

Parameters:

Name	Type	Required	Description
-----	-----	-----	-----
<code>docs</code>	<code>Document[]</code>	Yes	Array of documentation objects

Returns: `Map` — Cross-reference mapping

Changelog Functions

getTags(projectDir: string): Promise

Retrieves git tags from project repository.

Parameters:

Name	Type	Required	Description
-----	-----	-----	-----
<code>projectDir</code>	string	Yes	Path to git repository

Returns: `Promise` — Array of git tags

`getGitHubUrl(projectDir: string): Promise`

Extracts GitHub repository URL from git remote.

Parameters:

Name	Type	Required	Description
-----	-----	-----	-----
<code>projectDir</code>	string	Yes	Path to git repository

Returns: `Promise` — GitHub URL or null if not found

Diagram Functions

`DiagramGenerator` Class

Generates architecture diagrams using Mermaid.

Methods:

`generate(manifest: Manifest, type: DiagramType): Promise`

Generates diagram definition from manifest.

Parameters:

Name	Type	Required	Description
-----	-----	-----	-----
<code>manifest</code>	Manifest	Yes	Project manifest
<code>type</code>	DiagramType	Yes	Diagram type (flowchart, sequence, etc.)

Returns: `Promise` — Mermaid diagram definition

`switch(type: DiagramType): void` (inferred)

Switches the diagram generation mode.

Parameters:

Name	Type	Required	Description
-----	-----	-----	-----
<code>type</code>	DiagramType	Yes	New diagram type

`isMermaidCliAvailable(): Promise`

Checks if Mermaid CLI is installed and available.

Returns: `Promise` — True if Mermaid CLI is available **Example:**

```
import { isMermaidCliAvailable } from 'athena';

if (await isMermaidCliAvailable()) {
  console.log('Mermaid diagrams can be rendered');
}
```

Screenshot Functions**`stopDevServer(server: DevServer): void` (inferred)**

Stops a development server instance.

Parameters:

Name	Type	Required	Description
-----	-----	-----	-----
<code>server</code>	DevServer	Yes	Server instance to stop

`filterUsefulResults(results: Result[]): Result[]` (inferred)

Filters screenshot results to keep only useful captures.

Parameters:

Name	Type	Required	Description
-----	-----	-----	-----
<code>results</code>	Result[]	Yes	Array of screenshot results

Returns: `Result[]` — Filtered results

Data Structures

Manifest (inferred)

```
interface Manifest {
  project: string;
  framework: string;
  language: string;
  scanned: string;
  functions: FunctionDefinition[];
  classes: ClassDefinition[];
  scripts?: Record<string, string>;
  dependencies?: Record<string, string>;
}
```

FunctionDefinition (inferred)

```
interface FunctionDefinition {
  name: string;
  signature: string;
  location: string;
  async?: boolean;
}
```

ClassDefinition (inferred)

```
interface ClassDefinition {
  name: string;
  methods: string[];
  location: string;
}
```

Error Handling

Athena functions throw standard Node.js errors. Common error scenarios:

Error Type	Cause	Example
-----	-----	-----
<code>ENOENT</code>	Project directory not found	Invalid path passed to <code>scanProject()</code>
<code>EACCES</code>	Permission denied	Cannot read config file

APIError	Claude API error	Invalid API key or rate limit exceeded
ValidationError	Invalid configuration	Malformed .athenarc file

Example Error Handling:

```
try {
  const manifest = await scanProject('./my-project');
} catch (error) {

  if (error.code === 'ENOENT') {
    console.error('Project directory not found');
  } else if (error.name === 'APIError') {
    console.error('Claude API error:', error.message);
  } else {
    throw error;
  }
}
```

Rate Limiting

Claude API: Subject to Anthropic's rate limits based on your API tier. The `ClaudeClient` does not implement automatic retry logic (inferred). Consider implementing exponential backoff in your application code. **Recommended Practice:**

```
async function generateWithRetry(client, prompt, maxRetries = 3) {
  for (let i = 0; i < maxRetries; i++) {
    try {
      return await client.generate(prompt);
    } catch (error) {
      if (error.status === 429 && i < maxRetries - 1) {
        await new Promise(resolve => setTimeout(resolve, 2 * i * 1000));
        continue;
      }
      throw error;
    }
  }
}
```

Environment Variables

Variable	Required	Description
-----	-----	-----
ANTHROPIC_API_KEY	Yes	Claude API key for documentation generation

Configuration File

Athena looks for a configuration file in the project root (inferred: `.athenarc`, `.athena.json`, or `athena.config.js`).

Example Configuration:

```
{
  "exclude": ["node_modules", "dist", "coverage"],
  "include": ["src/**/*.ts"],
  "output": "docs",
  "model": "claude-3-5-sonnet-20241022"
}
```

Related Documentation

- [Readme](#)
- [Architecture](#)
- [Deployment](#)
- [Contributing](#)

Generated by Athena — © 2026 TheForge, LLC